Modelling a Live Metagame from Public Replays

A technical description of ABRA, the Automated Battle Replay Analyzer

Version 1.0 · Last updated 2026-07-22 Will Hooper · ABRA

> *This is a living document, updated in the same pass as any change to the code. New information is appended; a prior conclusion is not silently rewritten. See `CHANGELOG.md`.*

Abstract

ABRA is a live-data platform for competitive Pokémon Champions (Reg M-B). It continuously ingests public battle replays from Pokémon Showdown, extracts the durable facts of every game, and stores them as a growing dataset — the foundation for **modelling games and teams**. The near-term model is a metagame usage model; the larger goal is a simulator that learns which teams and which in-game choices win. This paper states the data model, the ingest architecture, the first analysis, and the limits. The governing design rule — *store raw, analyse on top* — is defended as the property that lets every model built on the data change without ever re-fetching. A team-preview engine (CHOMP) is one small, early consumer of the output; it is a proof of delivery, not the purpose.

1. The problem

Competitive VGC has many hard questions — which six to build against the meta, which four to bring, how a matchup tends to play out. Answering any of them well needs data on how the ladder *actually* plays, kept current as it

changes. Existing tools give static usage snapshots (Pikalytics) or per-game damage math (calculators); none continuously collects live games into a durable dataset that downstream models — team optimisers, game simulators, decision engines — can all build on. ABRA is that collection-and-storage layer. The bring-4 engine (CHOMP) is the first small model on top.

2. The data source

Pokémon Showdown exposes public replays through a stable API:

- `search.json?format=<id>` — recent public replays for a format, paginated (~51/page, ~100 pages).
- `search.json?user=<name>` — a specific player's public replays.
- `<id>.log` — the full battle protocol for one replay.

Every replay is a public artifact already attached to public usernames; ABRA reads nothing private and creates no accounts (see `SECURITY.md`).

3. The record — what we store per game

The extractor (`engine/durable-ingest.js`, `extract()`) turns one `.log` into one durable record:

Field	Meaning
`id`, `date`	replay id and upload time
`p1`, `p2`	`{name, rating, bot}` per player — rating from the `|player|` line, `bot` by name pattern
`six.p1/p2`	the revealed team of six (`|poke|` lines at preview)
`brought.p1/p2`	the Pokémon actually sent out (`|switch|` lines)
`lead.p1/p2`	the first two sent out
`sets`	per species, the moves / item / ability the replay *revealed*

Field	Meaning
`winner`	the winning name (`|win|`)

This is deliberately richer than any single analysis needs. That richness is the insurance behind the design rule in §5.

4. Ingest architecture

- **Concurrent fetch.** A fixed-size promise pool fetches logs in parallel. Measured throughput is ~200 logs/sec, so the full ~5,000 recent public replays ingest in about 25–30 seconds.
- **Incremental and idempotent.** The store is append-only JSON Lines keyed by replay id. Each run reads existing ids, requests recent pages, and appends only games not already present. Re-running never duplicates and never re-fetches stored games.
- **Continuous collection.** A scheduled GitHub Action runs the ingest hourly and commits the refreshed store and model, so the metagame model tracks the live ladder with no operator.

5. The governing rule — store raw, analyse on top

Every analysis and every filter runs over the stored records, not over the network. Concretely:

- The rating cutoff, the humans-only filter, and the "just my games" filter are all computed from the same store at read time.
- Changing how games are *placed* (a different rating tier, excluding bots, a new archetype tag) is a re-computation, **never** a re-pull.

The alternative — deciding the population at fetch time — would force a re-fetch every time the question changed. Storing raw makes the fetch a one-time cost and the analysis free to evolve. This is the single most important

property of the system.

6. The usage model

For a chosen population, `engine/analyze.js` computes per species:

```
teamRate  = teams containing the species        / teams
bringRate = games the species was brought        / games it was on the team
leadRate  = games the species was led            / games it was on the team
winRate   = games won when the species was brought / games it was brought
```

Species below a small sample floor (n ≥ 8) are dropped. The result is written to `data/meta-usage.json`, the model CHOMP reads.

6.1 A validated result

On 1,501 stored games (2,292 human teams), the raw ladder and the high-ladder cut (humans, rating ≥ 1300, 530 teams) differ meaningfully — the high-ladder cut lifts top-threat usage toward 50% and separates win rates (e.g. Kingambit 62%, Incineroar 65%). The top of the raw table independently reproduces a hand-curated threat list built from a separate ~141k-battle dataset (Basculegion, Whimsicott, Garchomp, Kingambit), which is external corroboration that the pipeline is correct.

7. Feeding CHOMP

CHOMP is a separate project — only the bring-4 / lead-2 engine. ABRA feeds it the ladder usage model (`meta-usage.json`) as **advisory intel**: what the opponent will *probably* bring and lead. CHOMP can sharpen that into a matchup-aware prediction — their best four against your six — but the **pick itself covers the opponent's whole six**. A player cannot know which four the opponent will bring, so CHOMP optimises for coverage over all six rather than gambling on a predicted four; ABRA's prediction informs the read, not the bet. Because CHOMP auto-updates from its repository, an improved ABRA model reaches the live plugin with no plugin change. This is the "gets smarter over time" loop, made concrete, and it is pinned by `CHOMP/tests/test-`

`meta-flow.js`.

8. The flywheel — where this is going

ABRA's data platform is stage one of a self-improving loop. Each stage feeds the next, and the last stage feeds the first:

1. **Collect** — continuously ingest live ladder games into the durable store. *(built and running)*
2. **Simulate** — build a near-perfect game simulator from that data: given two teams, model how the matchup tends to play out. *(roadmap)*
3. **Optimise teams** — run hypothetical teams against the real metagame threats in the simulator and let the search iterate on itself, converging on sixes that beat what the ladder actually brings. *(roadmap; the meta threat model that feeds it is built)*
4. **Play** — take an optimised team onto the ladder; CHOMP handles the bring-4 / lead-2 in each game. *(CHOMP built)*
5. **Feed back** — every opponent team you face is auto-added to the store, so playing the game grows the dataset. *(the ingest + store support this directly; the auto-add-on-play hook is roadmap)*

The loop closes: more games → a better simulator → better teams → more wins → more games. The value is not any single model but the flywheel — the system gets stronger the more it is used. This is why the design rule (*store raw, analyse on top*, §5) matters so much: every model in the loop is built on the same growing dataset, and none of them ever forces a re-pull.

Honest status. Stage 1 is built and validated (§4, §6.1). Stages 2 and 3 — the simulator and the self-iterating team optimiser — are not yet built; they are the reason ABRA exists and the next work. CHOMP (stage 4) is live. The paper does not claim the flywheel is spinning yet; it claims the foundation that makes it possible is in place.

9. Limitations

1. **Revealed sets are partial.** A Pokémon that never attacked reveals no moves; items/abilities appear only when they trigger. Set inference is a lower bound, not the full set.
2. **Descriptive, not predictive.** Usage and win rates describe the population; they are not a win-probability model.
3. **Population choice matters.** Bot and low-ladder games differ from high-ladder play. ABRA tags rating and bots so the consumer chooses the population; it does not decide for them.
4. **Store growth.** The append-only store grows without bound; pruning by date is future work.

10. References

1. Pokémon Showdown replay API — `replay.pokemonshowdown.com`.
2. `docs/reg-mb-threat-list.md` (CHOMP) — the independently curated threat list used for validation.
3. Keep a Changelog; Semantic Versioning; ASD-STE100; Diátaxis.

Companion documents. Slide deck · Technical documentation · Changelog